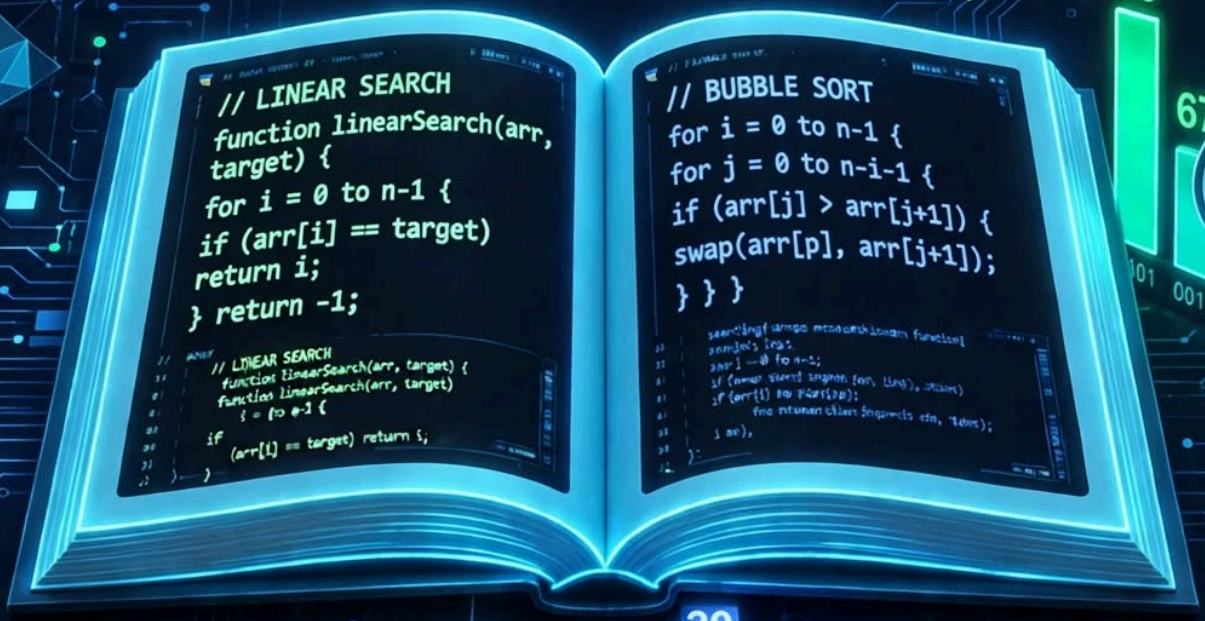
SMA NEGERI 4 YOGYAKARTA

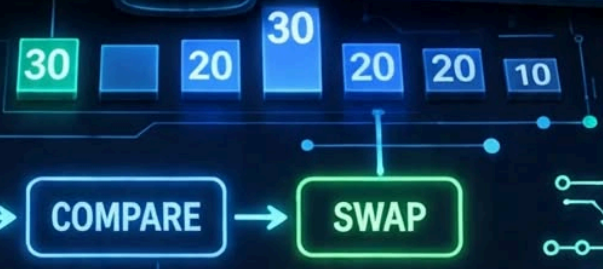
HANDBOOK

INFORMATIKA - ALGORITMA STANDAR

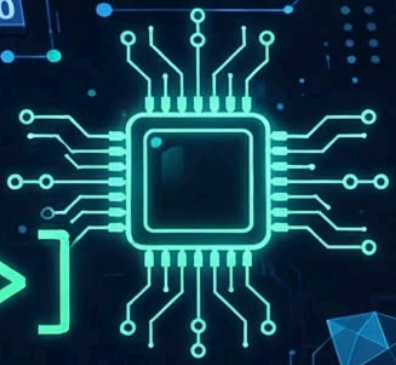
Searching dan Sorting - Panduan Lengkap











Disusun Oleh :
Ilyas Fandhi Anggara, S.Kom

Fase E • Kelas X • Semester Ganjil

Kurikulum Merdeka • Informatika SMA

KATA PENGANTAR

Puji syukur ke hadirat Tuhan Yang Maha Esa atas terselesaikannya buku saku ini. Buku ini disusun sebagai pendamping belajar Informatika bagi siswa kelas X SMA, khususnya dalam memahami **Algoritma Standar: Searching & Sorting** — pondasi penting dalam ilmu komputer dan pengolahan data.

Banyak siswa merasa algoritma itu abstrak dan rumit. Padahal, jika dipelajari lewat analogi sehari-hari — mencari nama di barisan, menyusun kartu remi, atau membuka kamus — algoritma justru terasa dekat dan logis.

Buku saku ini hadir dengan bahasa sederhana, contoh visual, serta tabel perbandingan yang memudahkan kamu memilih algoritma yang tepat. Setiap bagian dilengkapi analogi, contoh kerja, dan latihan mandiri.

Buku ini bukan untuk dibaca pasif. Ambil kertas, tulis langkah-langkahnya, dan coba simulasikan sendiri. Justru dari mencoba itulah pemahaman tumbuh.

Akhir kata, semoga buku kecil ini menjadi langkah awal yang menyenangkan untuk menapaki dunia algoritma dan berpikir komputasional. Selamat belajar, dan selamat berkarya.

Yogyakarta, 2025

Ilyas Fandhi Anggara, S.Kom

DAFTAR ISI

BAGIAN AWAL

— **Kata Pengantar** i

— **Daftar Isi** ii

— **Petunjuk Penggunaan Buku** 1

BAGIAN INTI — ALGORITMA SEARCHING & SORTING

Bagian 1 **Pengenalan Algoritma** 2
Definisi, analogi, dan dua pilar utama

Bagian 2 **Algoritma Searching** 4
Linear Search dan Binary Search

Bagian 3 **Algoritma Sorting** 8
Bubble, Insertion, dan Selection Sort

Bagian 4 **Sinergi Searching & Sorting** 12
Kombinasi keduanya dalam aplikasi nyata

Bagian 5 **Berpikir Komputasional** 13
4 pilar dalam searching & sorting

Bagian 6 **Cheat Sheet & Peta Konsep** 14
Ringkasan visual semua algoritma

BAGIAN AKHIR

— **Latihan Mandiri** 15

— **Kamus Istilah (Glosarium)** 16

— **Daftar Pustaka** 17

— **Profil Penyusun** 18

TIPS MEMBACA

Baca buku ini sambil praktik. Ambil kertas, tulis langkah-langkah algoritmanya, dan coba simulasi dengan data sederhana. Buku ini bukan untuk dihafal, tapi untuk dipraktikkan.

PETUNJUK PENGGUNAAN BUKU

Cara membaca buku saku ini agar hasil belajarmu maksimal.

I. **Pahami Analogi Sebelum Definisi**

Buku ini menyajikan analogi (barisan siswa, kamus, kartu remi) lebih dulu sebelum masuk ke definisi teknis.

II. **Ikuti Alur Data**

Setiap algoritma dijelaskan lewat contoh data konkret. Ikuti langkah demi langkah — dari data awal sampai data terurut atau target ditemukan.

III. **Gunakan Tabel Perbandingan**

Di akhir setiap bagian terdapat tabel ringkasan. Gunakan tabel ini untuk memutuskan algoritma mana yang paling tepat untuk kasus tertentu.

IV. **Hafalkan Kata Kunci**

Setiap algoritma punya "kata kunci" yang memudahkan ingatan — misalnya "Bagi Dua" untuk Binary Search, "Bandingkan & Tukar" untuk Bubble Sort.

V. **Kerjakan Latihan Mandiri**

Di akhir buku terdapat latihan. Bagian ini bukan pelengkap — justru di sinilah kamu benar-benar belajar.

SIMBOL YANG AKAN KAMU TEMUI

`[10, 20, 30]` = contoh data array (kumpulan angka).
`O(n)` = kompleksitas waktu (seberapa cepat algoritma bekerja).
Kata Kunci = frasa singkat untuk mengingat cara kerja algoritma.

BAGIAN

1

Pengenalan Algoritma

Langkah Sistematis Penyelesaian Masalah

BAGIAN 1: APA ITU ALGORITMA?

Algoritma adalah langkah-langkah sistematis dan logis untuk menyelesaikan masalah. Dalam pemrograman, **Algoritma Standar** digunakan untuk memproses dan mengelola data agar dapat diakses secara efisien.

ANALOGI SEDERHANA

Bayangkan kamu membuat telur goreng: Siapkan telur → Panaskan minyak → Pecahkan & goreng → Matang → Sajikan. Semua langkah **berurutan** dan **bertujuan** — itulah algoritma!

Dua Pilar Utama Algoritma Standar

1. SEARCHING (PENCARIAN)

Proses menemukan posisi atau keberadaan suatu data tertentu dari sekumpulan data.

```
Data: [30, 10, 50, 20, 40]
→ Cari angka 20
```

2. SORTING (PENGURUTAN)

Proses mengatur dan menyusun data berdasarkan aturan tertentu (kecil ke besar atau sebaliknya).

```
Data: [30, 10, 50, 20, 40]
→ Urutkan: [10, 20, 30, 40, 50]
```

INTINYA

Searching menjawab pertanyaan "Di mana data ini?". **Sorting** menjawab pertanyaan "Bagaimana data ini tersusun?". Keduanya adalah pondasi pengolahan data di dunia komputer.

Mengapa Algoritma Ini Penting?

Setiap hari, komputer memproses miliaran data. Tanpa algoritma yang efisien, aplikasi akan lambat dan boros sumber daya. Berikut contoh nyata di sekitar kita:

PENCARIAN DI GOOGLE

Google menggunakan algoritma pencarian canggih untuk menemukan halaman web relevan dari miliaran halaman dalam milidetik.

DAFTAR KONTAK HP

Kontak di ponselmu diurutkan secara alfabetis (Sorting) agar mudah dicari (Searching).

REKOMENDASI MARKETPLACE

Produk "paling relevan" diurutkan berdasarkan algoritma sorting dan pencarian dengan banyak kriteria.

PAPAN SKOR GAME

Leaderboard game diurutkan dari skor tertinggi ke terendah menggunakan algoritma sorting.

TAHUKAH KAMU?

Algoritma sorting tertua yang tercatat digunakan oleh bangsa Sumeria sekitar 3000 SM untuk mengurutkan data pertanian. Algoritma searching dan sorting sudah ada jauh sebelum komputer ditemukan!

BAGIAN

2

Algoritma Searching

Menemukan Data dengan Cepat

LINEAR SEARCH

👀 Satu per Satu | Cocok untuk Data Acak

Linear Search (pencarian linear) adalah algoritma pencarian paling sederhana. Ia memeriksa data **satu per satu** secara berurutan mulai dari awal sampai data yang dicari ditemukan atau seluruh data habis diperiksa.

ANALOGI & CONTOH KERJA

Mencari siswa **"Citra"** dalam barisan: Andi → Budi → Citra (Ketemu!). Kamu memeriksa satu per satu dari depan sampai menemukan Citra.

Simulasi Langkah demi Langkah

```
// Data awal
Data: [12, 25, 7, 30, 18] | Cari: 30

// Langkah pemeriksaan
12 ≠ 30 ❌  →  25 ≠ 30 ❌  →  7 ≠ 30 ❌  →  30 = 30 ✅
```

✅ KELEBIHAN

Mudah dibuat & tidak butuh data terurut.

❌ KEKURANGAN

Sangat lambat jika data sangat banyak.

KAPAN PAKAI LINEAR SEARCH?

Gunakan Linear Search ketika data **tidak terurut** atau jumlah data **relatif sedikit** (misalnya di bawah 100 item). Untuk data besar, pertimbangkan Binary Search.

BINARY SEARCH

✂️ Bagi Dua | ⚠️ Data Wajib Terurut!

Binary Search (pencarian biner) membagi ruang pencarian menjadi **dua bagian** secara berulang dengan mengecek nilai tengah. Jauh lebih cepat untuk data berukuran besar.

ANALOGI KAMUS & CARA KERJA

Mencari kata di kamus dengan membuka bagian tengah, lalu membuang setengah bagian yang tidak relevan. Kamu tidak membaca dari halaman pertama — kamu langsung ke tengah!

Simulasi Langkah demi Langkah

```
// Data harus sudah terurut
Data: [10, 20, 30, 40, 50, 60, 70] | Cari: 60

// Iterasi 1
Tengah: 40  →  60 > 40  →  Buang kiri!

// Iterasi 2
Sisa: [50, 60, 70]  →  Tengah: 60  →  60 = 60 ✅
```

✅ KELEBIHAN

Sangat cepat untuk data berukuran besar.

❌ SYARAT MUTLAK

Data harus sudah terurut terlebih dahulu.

PERBANDINGAN KECEPATAN

Untuk mencari 1 data di antara **1.000.000** data:
Linear Search: butuh hingga **1.000.000** pemeriksaan.
Binary Search: hanya butuh sekitar **20** pemeriksaan!

Tabel Perbandingan Searching

Aspek	Linear Search	Binary Search
Cara Mencari	Memeriksa satu per satu dari awal	Membagi ruang pencarian menjadi dua
Syarat Data	**Bebas (Tidak harus terurut)**	**Wajib Sudah Terurut**
Kecepatan Data Besar	Cenderung Lambat `O(n)`	**Sangat Cepat** `O(log n)`
Analogi Sehari-hari	Mencari siswa dalam barisan acak	Mencari kata di dalam kamus
Kata Kunci	👀 Satu per Satu	✂️ Bagi Dua

CARA MEMILIH

Jika data **sudah terurut** → pakai **Binary Search** (jauh lebih cepat). Jika data **masih acak** dan jumlahnya sedikit → pakai **Linear Search**. Jika data acak tapi sangat banyak → urutkan dulu dengan Sorting, baru Binary Search.

Studi Kasus: Mencari Buku di Perpustakaan

Sebuah perpustakaan memiliki 10.000 buku. Kamu ingin mencari buku berjudul "Algoritma Dasar". Mana algoritma yang tepat?

1 **Jika Buku Tidak Terurut (Linear Search)**

Petugas harus memeriksa rak demi rak, buku demi buku. Dalam kasus terburuk, 10.000 pemeriksaan. Lambat dan melelahkan.

2 **Jika Buku Terurut Alfabetis (Binary Search)**

Petugas langsung membuka rak tengah. Jika judul target < judul tengah, cari di rak kiri. Jika >, cari di rak kanan. Ulangi sampai ketemu. Hanya butuh sekitar **14 pemeriksaan** untuk 10.000 buku!

KESIMPULAN

Sorting (mengurutkan buku) adalah **investasi**. Sekali diurutkan, pencarian ke depannya menjadi sangat cepat. Inilah mengapa Searching dan Sorting sering dipakai bersama.

Latihan Kilat

LATIHAN 1

Data: `[8, 3, 15, 6, 12, 1]`. Cari angka **12** dengan Linear Search. Berapa langkah yang dibutuhkan?

LATIHAN 2

Data: `[2, 5, 8, 12, 16, 23, 38, 56, 72, 91]`. Cari angka **23** dengan Binary Search. Tuliskan urutan nilai tengah yang diperiksa!

BAGIAN

3

Algoritma Sorting

Menyusun Data dengan Logika

BUBBLE SORT

🫧 Bandingkan & Tukar

Bubble Sort membandingkan dua elemen yang bersebelahan. Jika urutannya salah, posisinya ditukar. Elemen terbesar perlahan bergerak ("mengapung") ke posisi paling kanan.

ANALOGI & CONTOH

Barisan siswa mengurutkan tinggi badan. Orang lebih tinggi di kiri bertukar ke kanan. Proses ini diulang sampai semua siswa terurut dari pendek ke tinggi.

Simulasi Langkah demi Langkah

```
// Data awal
[30, 10, 20, 40]

// Pass 1
30 > 10 → Tukar: [10, 30, 20, 40]
30 > 20 → Tukar: [10, 20, 30, 40]

// Hasil akhir
[10, 20, 30, 40] ✅
```

✅ KELEBIHAN

Sangat mudah dipahami dan diimplementasikan.

❌ KEKURANGAN

Lambat untuk data besar — kompleksitas `O(n²)`.

KATA KUNCI INGATAN

"Gelembung udara mengapung naik" — elemen terbesar perlahan naik ke posisi paling kanan, seperti gelembung di air.

INSERTION SORT

🃏 Geser & Sisipkan

Insertion Sort mengambil satu elemen data lalu membandingkannya dengan bagian yang sudah terurut. Elemen yang lebih besar digeser ke kanan, lalu data disisipkan pada posisi tepat.

ANALOGI & CONTOH

Menyusun kartu remi di tangan satu per satu dari kartu acak ke posisi yang benar. Kamu mengambil kartu baru, lalu menyisipkannya di antara kartu-kartu yang sudah tersusun.

Simulasi Langkah demi Langkah

```
// Mulai dengan satu elemen
Kartu: [30]

// Ambil 21, sisipkan sebelum 30
Dapat 21 → [21, 30]

// Ambil 17, geser 30 dan 21, sisipkan di depan
Dapat 17 → geser 30, geser 21 → [17, 21, 30] ✅
```

✅ KELEBIHAN

Efisien untuk data kecil atau data yang hampir terurut.

❌ KEKURANGAN

Kurang efisien untuk data besar yang benar-benar acak.

KATA KUNCI INGATAN

"Menyusun kartu tangan" — ambil satu kartu, geser yang lain, sisipkan di posisi tepat.

SELECTION SORT

🎯 Cari Terkecil & Tukar

Selection Sort mencari nilai terkecil dari seluruh bagian yang belum terurut, kemudian menukarkannya langsung ke posisi paling depan.

ANALOGI & CONTOH

Guru memilih nilai terkecil dari tumpukan ujian, menaruhnya di depan, lalu mengulangi sisa lembaran. Setiap kali, ia mencari yang paling kecil dari yang tersisa.

Simulasi Langkah demi Langkah

```
// Data awal
[30, 10, 40, 20]

// Cari terkecil dari seluruh data
Terkecil: 10  →  Tukar ke depan
[10, 30, 40, 20]

// Cari terkecil dari sisa [30, 40, 20]
Terkecil: 20  →  Tukar ke posisi kedua
[10, 20, 40, 30]

// Lanjutkan hingga terurut
[10, 20, 30, 40] ✅
```

✅ KELEBIHAN

Jumlah penukaran relatif sedikit dibanding Bubble Sort.

❌ KEKURANGAN

Tetap `O(n²)` — lambat untuk data besar.

KATA KUNCI INGATAN

"Memilih nilai terendah satu per satu" — seperti memilih pemain terbaik dari daftar, lalu yang terbaik kedua, dan seterusnya.

Ringkasan Metode Sorting

Algoritma	Kata Kunci Utama	Cara Kerja Ringkas	Analogi Ingatan
Bubble Sort 🫧	**Bandingkan & Tukar**	Bandingkan 2 elemen berdampingan, tukar jika salah urut	Gelembung udara mengapung naik
Insertion Sort 🃏	**Ambil, Geser, Sisipkan**	Ambil data, geser elemen lain, sisipkan di posisi tepat	Menyusun kartu tangan
Selection Sort 🎯	**Cari Terkecil & Tukar**	Cari data terkecil dari sisa array, tukar ke depan	Memilih nilai terendah satu per satu

CARA MEMILIH ALGORITMA SORTING

Data kecil (< 50) → Bubble / Insertion / Selection (sama saja).
Data hampir terurut → Insertion Sort (paling efisien).
Ingin sedikit penukaran → Selection Sort.
Data sangat besar → Merge Sort / Quick Sort (di luar cakupan buku ini).

BAGIAN

4

Sinergi Searching & Sorting

Bekerja Sama Menyelesaikan Masalah

MENGGABUNGKAN KEDUANYA

Searching dan Sorting sering kali digunakan secara **bersamaan** dalam aplikasi nyata. Sorting menjadi "persiapan" agar Searching bisa bekerja maksimal.

CONTOH KASUS LENGKAP

Data acak: `[40, 10, 70, 20, 50, 30]`. Kita ingin mencari angka **50** menggunakan **Binary Search**.

1 **Langkah 1 — Sorting (Persiapan)**

Urutkan data terlebih dahulu dengan salah satu algoritma sorting. Hasilnya: `[10, 20, 30, 40, 50, 70]`.

2 **Langkah 2 — Searching (Eksekusi)**

Gunakan Binary Search untuk menemukan posisi angka 50 secara cepat. Karena data sudah terurut, Binary Search bisa membagi dua ruang pencarian berulang kali.

3 **Hasil**

Angka 50 ditemukan pada posisi ke-5 (indeks 4) dengan hanya **2–3 pemeriksaan**, bukan 5 pemeriksaan seperti Linear Search.

PELAJARAN PENTING

Sorting adalah **investasi waktu**. Sekali kamu mengurutkan data, pencarian ke depannya menjadi jauh lebih efisien. Inilah mengapa database dan sistem pencarian selalu menyimpan data dalam keadaan terurut.

Contoh Nyata di Dunia Kerja

E-COMMERCE

Produk diurutkan berdasarkan harga (Sorting), lalu pengguna mencari produk tertentu (Searching).

PERPUSTAKAAN DIGITAL

Buku diurutkan berdasarkan judul (Sorting), lalu pengunjung mencari buku spesifik (Searching).

SISTEM AKADEMIK

Nilai siswa diurutkan dari tertinggi (Sorting), lalu dicari siswa dengan nilai tertentu (Searching).

MEDIA SOSIAL

Konten diurutkan berdasarkan relevansi (Sorting), lalu pengguna mencari topik tertentu (Searching).

BAGIAN

5

Berpikir Komputasional

4 Pilar dalam Searching & Sorting

4 PILAR BERPIKIR KOMPUTASIONAL

Algoritma Searching dan Sorting adalah contoh nyata penerapan **4 pilar berpikir komputasional**. Mari kita bedah satu per satu.

🧩 DEKOMPOSISI

Memecah proses pencarian/pengurutan menjadi langkah kecil: data input, target, dan metode.

Contoh: Binary Search dipecah jadi "cari tengah → bandingkan → buang setengah → ulangi".

🔍 PENGENALAN POLA

Melihat pola: data terurut cocok dengan Binary Search; dua data bersebelahan cocok dengan Bubble Sort.

Contoh: Jika data sudah terurut, langsung pilih Binary Search — itu polanya.

💡 ABSTRAKSI

Fokus pada angka/target, mengabaikan informasi yang tidak relevan dengan pencarian.

Contoh: Saat mencari angka 50, kita tidak peduli warna sel atau nama pemilik data — hanya nilainya.

📐 ALGORITMA

Menyusun instruksi berurutan: periksa → bandingkan → tukar/sisipkan.

Contoh: Langkah-langkah Bubble Sort ditulis berurutan dan diulang sampai data terurut.

MENGAPA INI PENTING?

Dengan memahami 4 pilar ini, kamu tidak hanya menghafal algoritma — kamu belajar **cara berpikir** untuk menyelesaikan masalah apa pun, bukan hanya soal searching dan sorting.

BAGIAN

6

Cheat Sheet & Peta Konsep

Ringkasan Cepat Semua Algoritma

PETA KONSEP RINGKAS

Gunakan peta konsep ini sebagai pengingat cepat. Hafalkan kata kuncinya, dan kamu akan selalu ingat cara kerja setiap algoritma.

LINEAR
👀 Satu per satu

BINARY
✂️ Bagi dua

BUBBLE
🫧 Banding & Tukar

INSERTION
🃏 Geser & Sisip

SELECTION
🎯 Cari Terkecil

Tabel Kompleksitas Waktu

Algoritma	Kompleksitas	Kapan Digunakan
Linear Search	`O(n)`	Data acak, jumlah sedikit
Binary Search	`O(log n)`	Data terurut, jumlah besar
Bubble Sort	`O(n²)`	Data sangat kecil, untuk belajar
Insertion Sort	`O(n²)`	Data hampir terurut
Selection Sort	`O(n²)`	Ingin sedikit penukaran

TIPS MENGHAFAL

Searching: "Linear itu pelan tapi bebas; Binary itu cepat tapi harus terurut."

Sorting: "Bubble = gelembung naik; Insertion = kartu tangan; Selection = pilih yang terkecil."

LATIHAN MANDIRI

Uji pemahamanmu dengan latihan berikut. Ambil kertas dan tulis langkah-langkahnya secara manual.

LATIHAN 1 — LINEAR SEARCH

Data: `[14, 7, 22, 9, 31, 5]`. Cari angka **9** dengan Linear Search. Tuliskan urutan pemeriksaan sampai angka ditemukan. Berapa langkah yang dibutuhkan?

LATIHAN 2 — BINARY SEARCH

Data: `[3, 8, 12, 17, 25, 33, 41, 56, 68, 79]`. Cari angka **41** dengan Binary Search. Tuliskan nilai tengah yang diperiksa pada setiap iterasi.

LATIHAN 3 — BUBBLE SORT

Urutkan data `[8, 3, 5, 1, 9]` dengan Bubble Sort. Tuliskan kondisi array setelah setiap pass.

LATIHAN 4 — INSERTION SORT

Urutkan data `[6, 2, 8, 4, 1]` dengan Insertion Sort. Tuliskan kondisi array setelah setiap penyisipan.

LATIHAN 5 — SELECTION SORT

Urutkan data `[9, 4, 7, 2, 5]` dengan Selection Sort. Tuliskan nilai terkecil yang dipilih pada setiap iterasi.

LATIHAN 6 — KOMBINASI

Data: `[25, 10, 40, 15, 35]`. Urutkan dengan algoritma sorting pilihanmu, lalu cari angka **35** dengan Binary Search. Tuliskan semua langkahnya.

LATIHAN 7 — ANALISIS

Mengapa Binary Search lebih cepat daripada Linear Search? Jelaskan dengan bahasa sendiri, dan sebutkan syarat utama yang harus dipenuhi.

KAMUS ISTILAH

Istilah teknis yang wajib kamu pahami selama belajar algoritma.

Algoritma
Langkah-langkah sistematis dan logis untuk menyelesaikan masalah.

Array
Kumpulan data yang tersusun dalam urutan tertentu, mis. [10, 20, 30].

Binary Search
Algoritma pencarian dengan membagi data menjadi dua bagian secara berulang.

Bubble Sort
Sorting dengan membandingkan dua elemen bersebelahan dan menukarnya jika salah urut.

Elemen
Satu unit data dalam array, mis. angka 30 dalam [30, 10, 20].

Insertion Sort
Sorting dengan mengambil elemen lalu menyisipkannya ke posisi tepat.

Kompleksitas
Ukuran seberapa efisien algoritma, biasanya ditulis O(n), O(log n), $O(n^2)$.

Linear Search
Pencarian dengan memeriksa data satu per satu dari awal.

Pass
Satu putaran lengkap dalam algoritma sorting.

Pencarian
Proses menemukan posisi atau keberadaan data tertentu (Searching).

Pengurutan
Proses menyusun data berdasarkan aturan tertentu (Sorting).

Selection Sort
Sorting dengan mencari nilai terkecil dari sisa data dan menukarnya ke depan.

Sorting
Proses mengatur data menjadi urutan tertentu (naik/turun).

Tukar (Swap)
Menukar posisi dua elemen dalam array.

DAFTAR PUSTAKA

Sedgewick, R. (1990). *Algorithms in C*. Addison-Wesley. (Membahas komprehensif algoritma sorting, searching, dan analisisnya.)

Knuth, D. E. (1998). *The Art of Computer Programming, Volume 3: Sorting and Searching* (2nd ed.). Addison-Wesley. (Referensi klasik dan paling komprehensif untuk sorting & searching.)

Jagtap, A., & Mali, A. (2021). Searching and Sorting Techniques. Dalam *Data Structures using C*. Taylor & Francis. https://doi.org/10.1201/9781003105800-7

Hartono, S.Pd., M.T.I. (n.d.). *Algoritma Pengurutan (Sorting) dan Implementasinya di Python (bubble, insertion, selection)*. Universitas Muhammadiyah Kotabumi. https://lmsspada.kemdikbud.go.id/

Biro Pengembangan Inovasi Dan Karir Universitas Medan Area. (2025). *Algoritma Pencarian dan Pengurutan dari Linear Search hingga Quick Sort*. https://bpika.uma.ac.id/2025/06/10/algoritma-pencarian-dan-pengurutan-dari-linear-search-hingga-quick-sort/

Kementerian Pendidikan, Kebudayaan, Riset, dan Teknologi. (2021). *Informatika untuk SMA/SMK Kelas X*. Pusat Kurikulum dan Perbukuan.

Carnegie Mellon University. (n.d.). *Recursion II and Search Algorithms*. https://www.cs.cmu.edu/~15110/practice/unit2/

CATATAN

Referensi di atas digunakan sebagai dasar penyusunan materi dan contoh. Untuk latihan interaktif, silakan kunjungi e-portfolio penulis.

PROFIL PENYUSUN



Ilyas Fandhi Anggara, S.Kom

Guru Informatika • SMA Negeri 4 Yogyakarta

Seorang pendidik yang percaya bahwa setiap siswa mampu belajar algoritma asalkan diberi pendekatan yang tepat. Aktif mengajar Informatika kelas X dengan metode praktik langsung dan pemanfaatan teknologi digital.

Email ilyasfandhianggara@gmail.com
Website e-portofolio-fandhi.vercel.app
Lokasi Yogyakarta, Indonesia

> *"Algoritma bukan tentang menghafal.*
> *Ini tentang cara berpikir."*
>
> — ILYAS FANDHI ANGGARA

✦ ✦ ✦

Saatnya Berpikir Komputasional

Semakin rapi langkahmu, semakin efisien algoritmamu. Selamat belajar Searching & Sorting!